

Exercise 11

Advanced Methods for Regression and Classification

Kyzer Gerez

14.01.2026

Take the data set `german.credit` from the package `fairml` (see help page). The variable `Credit_risk` is the grouping variable, and it indicates whether the persons were successful with their loan application or not. The “BAD” clients are particularly interesting for us, but they are in the minority (= imbalanced classes)

For the following tasks, select randomly about 2/3 of the observations from each class as training data, and the rest as test data.

```
library(fairml); data(german.credit)
library(rpart)
library(rpart.plot)

## Warning: package 'rpart.plot' was built under R version 4.4.3

library(randomForest)
library(caret)

set.seed(12427543)

# separate the two classes
credit.BAD <- german.credit[german.credit$Credit_risk == 'BAD',]
credit.GOOD <- german.credit[german.credit$Credit_risk == 'GOOD',]

# get sample sizes for each class
samp.size.BAD <- floor(2/3*nrow(credit.BAD))
samp.size.GOOD <- floor(2/3*nrow(credit.GOOD))

# create train index
train.indx <- sample(as.numeric(rownames(credit.BAD)), size=samp.size.BAD)
train.indx <- c(train.indx, sample(as.numeric(rownames(credit.GOOD)), samp.size.GOOD))

# create stratified train and test splits
train <- german.credit[train.indx,]
test <- german.credit[-train.indx,]
```

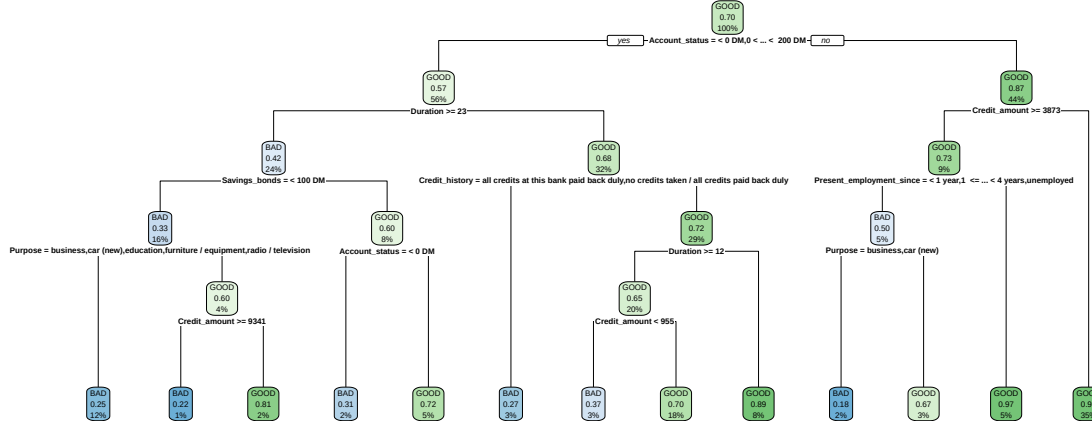
1. Classification trees: function `rpart()` from the R package `rpart`

(a) Compute an initial tree T_0 (see `help(rpart)` or lecture notes).

```
mod.T_0 <- rpart(Credit_risk~., data=train, method='class', xval=20)
```

(b) Visualize the tree with the function `rpart.plot()` from the package `rpart.plot`, and try to interpret the results.

```
rpart.plot(mod.T_0)
```



Each node shows the predicted class and the percentage of observations in the node. At each branch, if the conditions are met, then take the left subtree. Else, take the right subtree. The middle number in each node represents the probability that an observation that falls to this node has a good credit risk i.e. nodes labelled 'BAD' will have low values.

For the above tree, the first branch checks the account status. If it's less than 200 DM, the left subtree is taken. At this next level, 56% of observations go to the left subtree, while the rest take the right. This process repeats until a leaf node is reached.

- (c) Predict the class variable for the test set (see `help(predict.rpart)` or lecture notes). Show the confusion table and report the balanced accuracy.

```
# predict
pred.T_0 <- predict(mod.T_0, newdata=test, type='class')

# create confusion matrix and report required output
confusionMatrix.T_0 <- confusionMatrix(as.factor(pred.T_0),
                                       as.factor(test$Credit_risk))

confusionMatrix.T_0$table

##           Reference
## Prediction BAD GOOD
##      BAD    44   33
##      GOOD   56  201

confusionMatrix.T_0$byClass['Balanced Accuracy']

## Balanced Accuracy
##           0.6494872
```

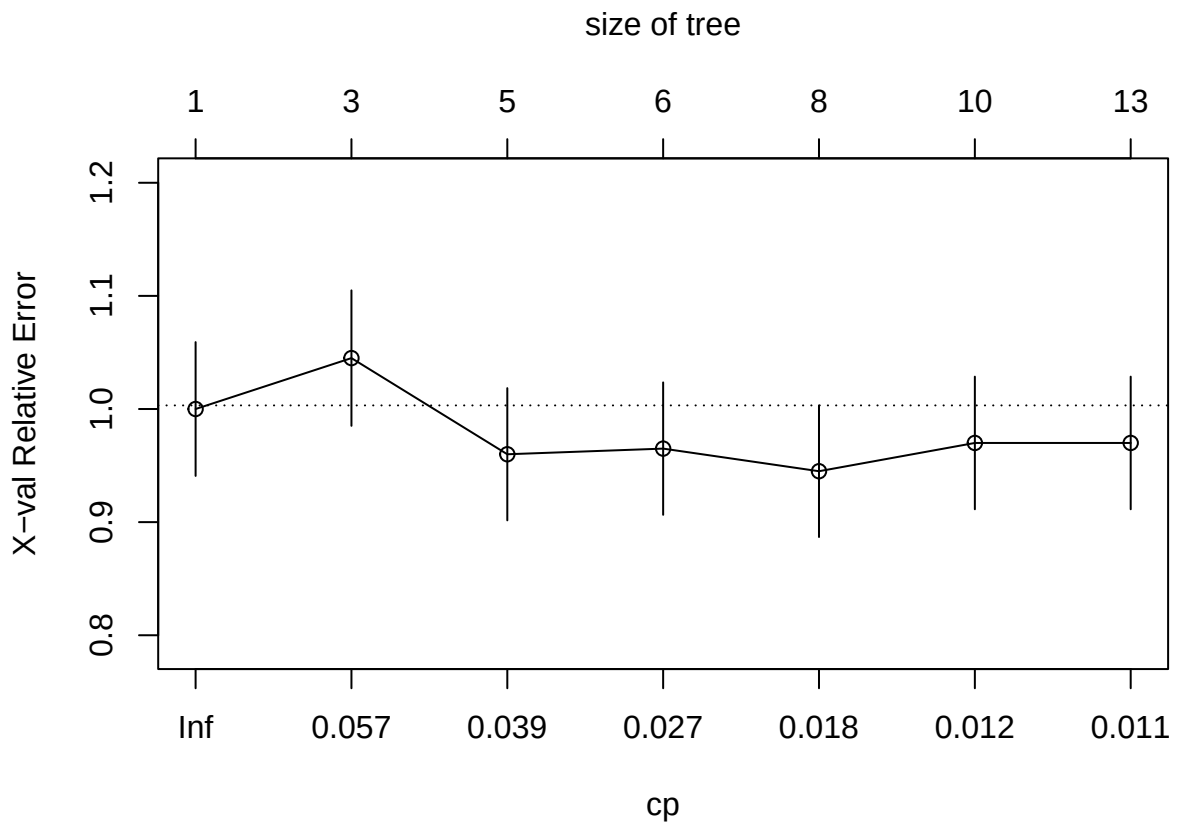
- (d) Show and interpret results of cross-validation obtained by using `plotcp()`. What is the optimal tree complexity?

```
printcp(mod.T_0)

##
## Classification tree:
## rpart(formula = Credit_risk ~ ., data = train, method = "class",
##       xval = 20)
##
## Variables actually used in tree construction:
## [1] Account_status      Credit_amount        Credit_history
```

```
## [4] Duration                Present_employment_since Purpose
## [7] Savings_bonds
##
## Root node error: 200/666 = 0.3003
##
## n= 666
##
##      CP nsplit rel error xerror      xstd
## 1 0.065000      0    1.000  1.000 0.059148
## 2 0.050000      2    0.870  1.045 0.059878
## 3 0.030000      4    0.770  0.960 0.058448
## 4 0.025000      5    0.740  0.965 0.058539
## 5 0.012500      7    0.690  0.945 0.058173
## 6 0.011667      9    0.665  0.970 0.058628
## 7 0.010000     12    0.630  0.970 0.058628
```

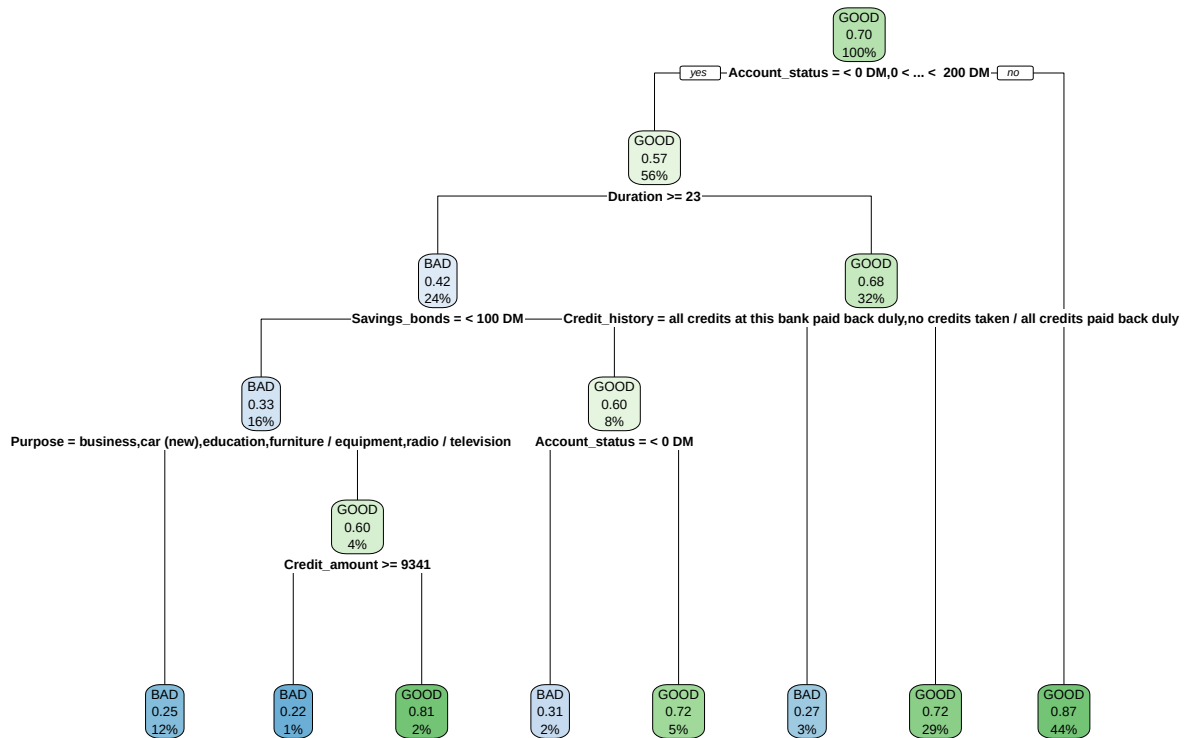
```
plotcp(mod.T_0)
```



The optimal tree complexity appears to be 0.0125.

(e) Prune the tree T_0 to the optimal complexity using `prune()`. Visualize and interpret the results.

```
mod.T_0.pruned <- prune(mod.T_0, cp=0.0125, method='class')
rpart.plot(mod.T_0.pruned)
```



The pruned tree is of course simpler than the initial tree. The same variables are used. However, there are only 7 splits compared to 12 for the initial tree.

- (f) Predict the class variable for the test set, show the confusion table, and report the balanced accuracy. Do we observe any improvement?

```
# predict
pred.T_0.pruned <- predict(mod.T_0.pruned, newdata=test, type='class')

# create confusion matrix and report required output
confusionMatrix.T_0.pruned <- confusionMatrix(as.factor(pred.T_0.pruned),
                                              as.factor(test$Credit_risk))
confusionMatrix.T_0.pruned$table

##           Reference
## Prediction BAD GOOD
##          BAD   35   22
##          GOOD  65  212

confusionMatrix.T_0.pruned$byClass['Balanced Accuracy']

## Balanced Accuracy
##           0.6279915
```

Overall, the pruned tree led to worse results in terms of the balanced accuracy. It was better in predicting the “GOOD” class, but performed much worse in predicting the ‘BAD’ class, which led to a lower balanced accuracy.

- (g) A simple way to improve the balanced accuracy could be to make use of the argument `weights` within `rpart()`. Try it out and report the results.

The approach I will take is to find the proportions of each class and use the proportion for the opposite class as the weight.

```

# create a weight vector based on the Credit_risk values
weight.BAD <- nrow(credit.GOOD)/nrow(german.credit)
weight.GOOD <- nrow(credit.BAD)/nrow(german.credit)
weights_vec <- ifelse(train$Credit_risk == "BAD", weight.BAD, weight.GOOD)

# create weighted tree
mod.T.weight <- rpart(Credit_risk~ ., data=train, weights=weights_vec)

# predict
pred.T.weight <- predict(mod.T.weight, newdata=test, type='class')

# create confusion matrix and report required output
confusionMatrix.T.weight <- confusionMatrix(as.factor(pred.T.weight),
                                             as.factor(test$Credit_risk))
confusionMatrix.T.weight$table

##           Reference
## Prediction BAD GOOD
##      BAD    59    63
##      GOOD   41   171

confusionMatrix.T.weight$byClass['Balanced Accuracy']

## Balanced Accuracy
##           0.6603846

```

The weights used led to an improved balanced accuracy. The prediction for the “BAD” class improved at the cost of the “GOOD” class. But this is tolerable since the “BAD” clients are the ones of interest.

2. Random Forests: function `randomForest()` from the R package `randomForest`

- (a) Use Random Forests to classify the training data and predict the class variable for the test data. Report the resulting confusion table and the balanced accuracy.

```

# train random forest on default parameters
mod.RF_0 <- randomForest(Credit_risk~ ., data=train)

# predict
pred.RF_0 <- predict(mod.RF_0, test, type='class')

# create confusion matrix and report required output
confusionMatrix.RF_0 <- confusionMatrix(as.factor(pred.RF_0),
                                         as.factor(test$Credit_risk))
confusionMatrix.RF_0$table

##           Reference
## Prediction BAD GOOD
##      BAD    36    22
##      GOOD   64   212

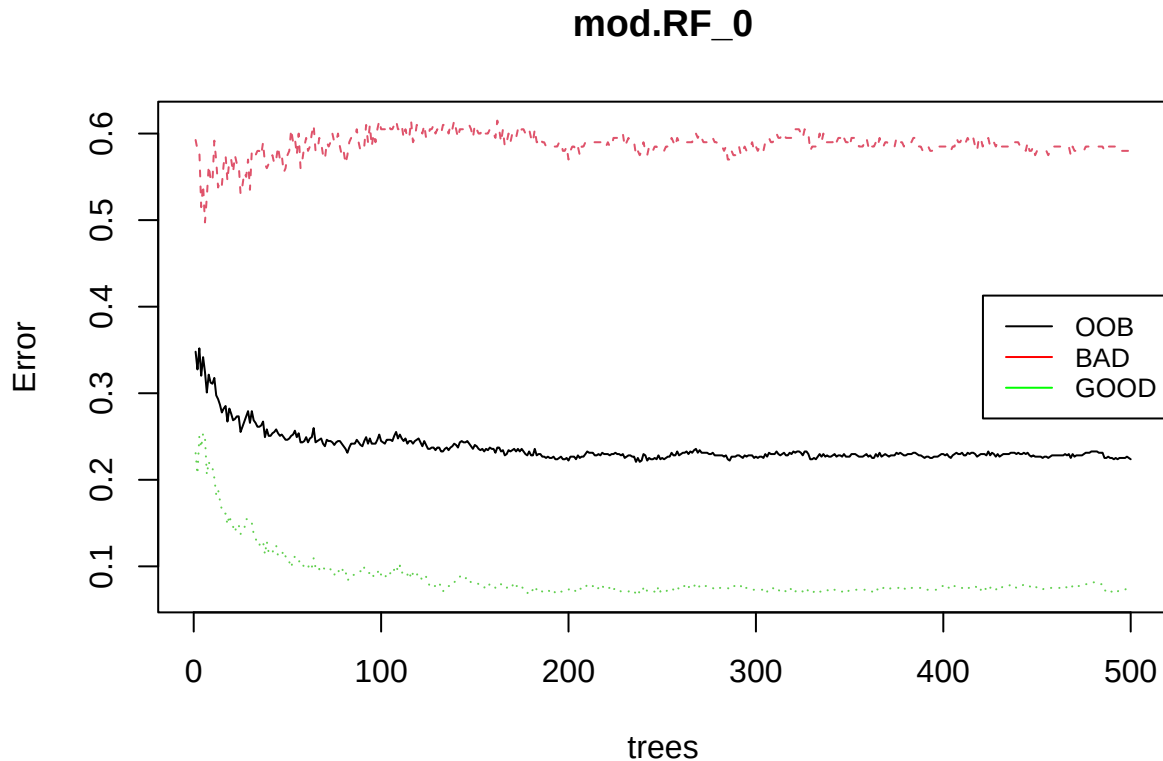
confusionMatrix.RF_0$byClass['Balanced Accuracy']

## Balanced Accuracy
##           0.6329915

```

- (b) Plot the result object with `plot()` and interpret the plot.

```
plot(mod.RF_0)
legend("right",
      legend = c("OOB", levels(train$Credit_risk)),
      col = c("black", "red", "green"),
      lty = 1,
      cex = 0.8)
```



```
mod.RF_0[["err.rate"]][1:10,]
```

	OOB	BAD	GOOD
## [1,]	0.3480000	0.5925926	0.2307692
## [2,]	0.3275000	0.5813953	0.2066421
## [3,]	0.3518887	0.5806452	0.2500000
## [4,]	0.3202147	0.5149701	0.2372449
## [5,]	0.3417085	0.5409836	0.2536232
## [6,]	0.3246753	0.4973262	0.2494172
## [7,]	0.3007874	0.5265957	0.2058166
## [8,]	0.3214838	0.5631579	0.2210066
## [9,]	0.3123077	0.5468750	0.2139738
## [10,]	0.3109756	0.5440415	0.2138229

The y-axis is error rates and x-axis is the number of trees. The black line represents the overall error rate, the red line is the error rate for the “BAD” class, and the green line is the error rate for the “GOOD” class.

The plot shows that the overall and “GOOD” error rates decrease with an increasing number of trees. However, the opposite behavior can be seen with for the error rate of the “BAD” class.

An inspection of the error rates show that the minimum error rate for the “BAD” class is achieved with only 2 trees.

(c) Try to improve the balanced accuracy with different strategies:

- Modify the parameter `sampsiz` in the `randomForest()` function. What is it doing?

This parameter maps to the bootstrap sample size. This means that it controls how many observations are randomly sampled from each node during the construction of the random forest model. With `replace=TRUE` (default), `sampsiz=nrow(x)`.

To determine the value to set the parameter to, I tested different of x for `ceiling(x*nrow(train))` in 0.1 increments from 0.1 to 1.0. The value that maximizes the balanced accuracy is 0.5.

```
# train RF
set.seed(12427543)
mod.RF.sampsiz <- randomForest(Credit_risk~ ., data=train,
                               sampsiz=ceiling(0.5*nrow(train)))

# predict
pred.RF.sampsiz <- predict(mod.RF.sampsiz, test, type='class')

# create confusion matrix and report required output
confusionMatrix.RF.sampsiz <- confusionMatrix(as.factor(pred.RF.sampsiz),
                                              as.factor(test$Credit_risk))
confusionMatrix.RF.sampsiz$table
```

```
##           Reference
## Prediction BAD GOOD
##          BAD   36   24
##          GOOD  64  210
```

```
confusionMatrix.RF.sampsiz$byClass['Balanced Accuracy']
```

```
## Balanced Accuracy
##           0.6287179
```

- Modify the parameter `classwt` in the `randomForest()` function. What is it doing?

This has a similar function as the `weights` parameter for `rpart()` in that it can be used to deal with imbalanced classes. In the below approach, I use a similar weight vector as in part 1g.

```
set.seed(12427543)

# train RF
mod.RF.classwt <- randomForest(Credit_risk~ ., data=train,
                               classwt=c(weight.BAD, weight.GOOD))

# predict
pred.RF.classwt <- predict(mod.RF.classwt, test, type='class')

# create confusion matrix and report required output
confusionMatrix.RF.classwt <- confusionMatrix(as.factor(pred.RF.classwt),
                                              as.factor(test$Credit_risk))
confusionMatrix.RF.classwt$table
```

```
##           Reference
## Prediction BAD GOOD
##          BAD   34   24
##          GOOD  66  210
```

```
confusionMatrix.RF.classwt$byClass['Balanced Accuracy']
```

```
## Balanced Accuracy
```

```
##          0.6187179
```

- Modify the parameter `cutoff` in the `randomForest()` function. What is it doing?

This parameter controls the number of votes needed to determine the class of an observation. By default, the majority vote wins i.e. $1/k$ where k is the number of classes.

For the cell below, I tested different combinations of the cutoff vector in increments of 0.05 e.g. `c(0.05, 0.95)`, `c(0.10, 0.90)`, ... The best results were achieved with a cutoff vector of `c(0.35, 0.65)` favoring the “BAD” class.

```
set.seed(12427543)

# train RF
mod.RF.cutoff <- randomForest(Credit_risk~ ., data=train,
                              cutoff=c(0.35, 0.65))

# predict
pred.RF.cutoff <- predict(mod.RF.cutoff, test, type='class')

# create confusion matrix and report required output
confusionMatrix.RF.cutoff <- confusionMatrix(as.factor(pred.RF.cutoff),
                                              as.factor(test$Credit_risk))

confusionMatrix.RF.cutoff$table
```

```
##          Reference
## Prediction BAD GOOD
##          BAD   67   57
##          GOOD  33  177
```

```
confusionMatrix.RF.cutoff$byClass['Balanced Accuracy']
```

```
## Balanced Accuracy
##          0.7132051
```

Which approach leads to the overall best solution?

Overall, changing the `cutoff` parameter gave the best solution by far. It resulted in a balanced accuracy of 71.3% compared to 61.9% for the default random forest model.

Modifying `sampsiz` led to a marginal improvement. It correctly predicted two more “BAD” observations, which is an increase in the balanced accuracy of 1% (absolute) over the default random forest model.

The weights chosen for the `classwt` yielded the same results as the default random forest model.

- Run the function `randomForest()` based on the best strategy from above, with the additional option `importance=TRUE`, and plot the result object with `plot()` and `varImpPlot()`. How can you interpret these plots?

```
set.seed(12427543)

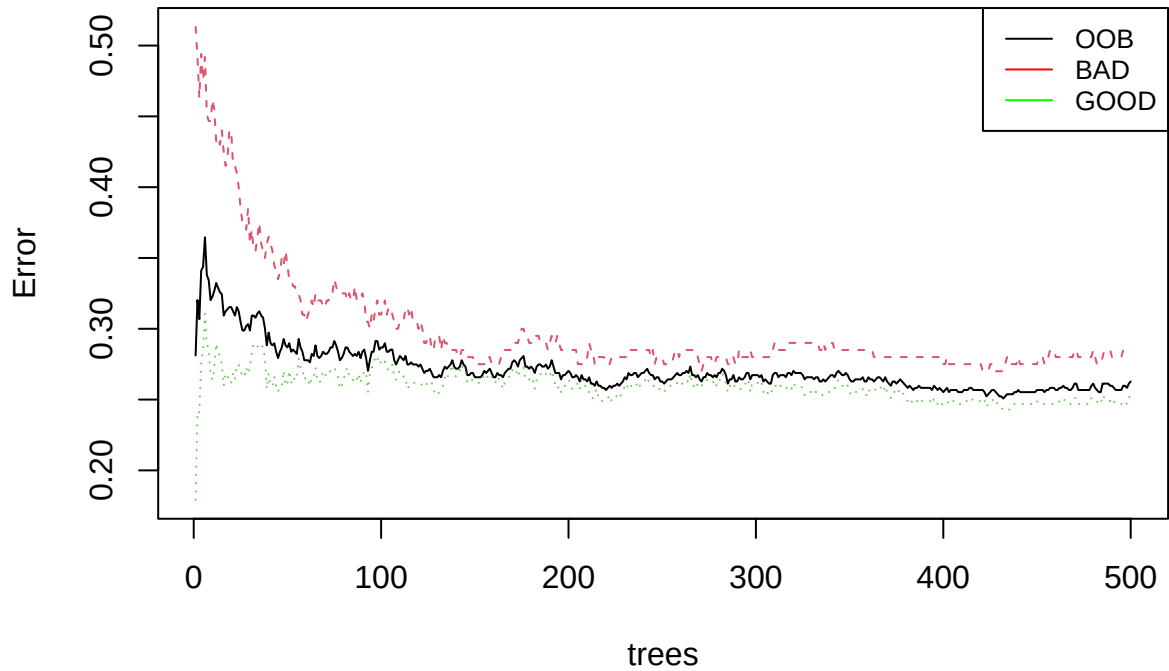
# train RF
mod.RF.best <- randomForest(Credit_risk~ ., data=train,
                            cutoff=c(0.35, 0.65),
                            importance=TRUE)

plot(mod.RF.best)
legend("topright",
      legend = c("OOB", levels(train$Credit_risk)),
```



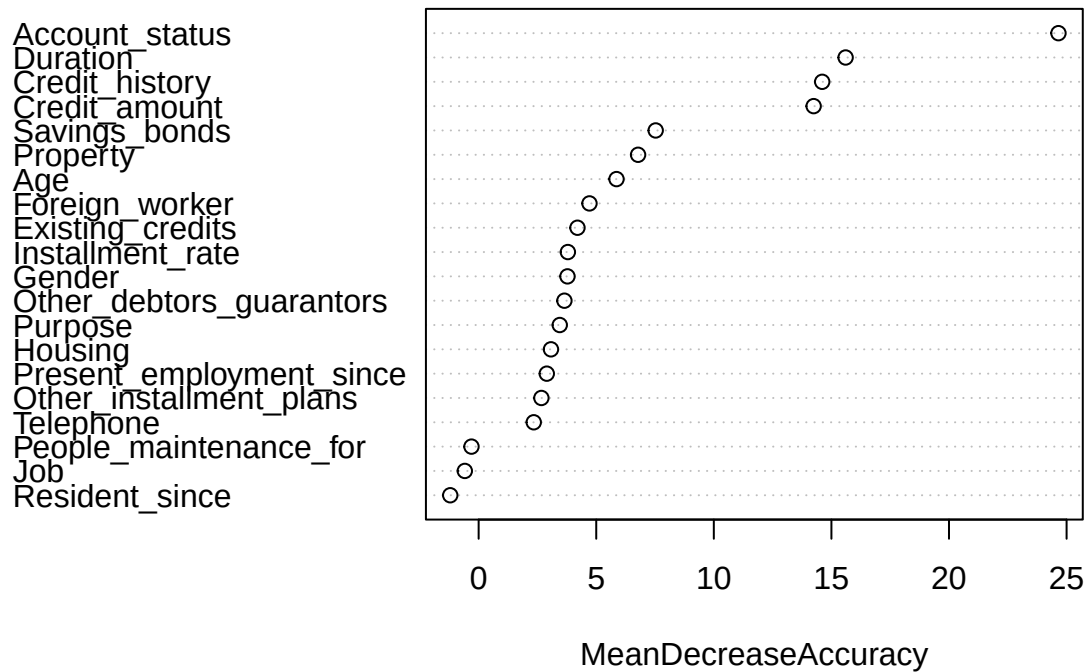
```
col = c("black", "red", "green"),
lty = 1,
cex = 0.8)
```

mod.RF.best

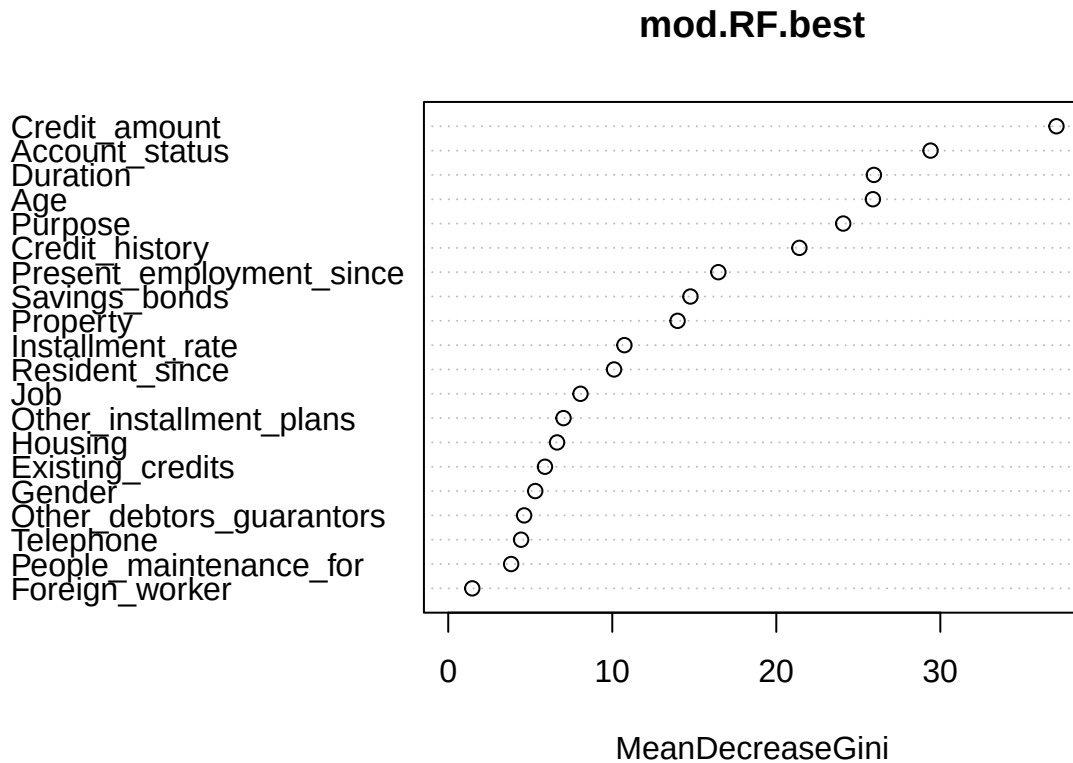


```
varImpPlot(mod.RF.best, type=1)
```

mod.RF.best



```
varImpPlot(mod.RF.best, type=2)
```



The results for `plot()` shows much better behavior than that from the default random forest model in that increasing the number of trees decreases the error rate for both classes. The error rate for the “GOOD” class does settle at a higher value than in the default model, but this is acceptable since the performance on the “BAD” class is greatly improved.

The function `varImpPlot()` displays the importance of variables based on two different measures. The top plot is based on the mean decrease in accuracy. The bottom plot uses the mean decrease in node impurity.

The lecture notes explain that the Gini index or node impurity is preferred over misclassification error. Given this recommendation, the top three most important variables are **Credit_amount**, **Account_status**, and **Duration**.

The only disagreement with the misclassification error method in the top-3 is that it swaps **Credit_amount** for **Credit_history**.